Лекция 2. Визуализация данных. Представление результатов исследования

Введение

Визуализация данных — это не просто построение графиков, а важнейший инструмент анализа и коммуникации [1]. Человек воспринимает визуальную информацию значительно быстрее, чем текст или таблицы: один график может заменить страницу чисел. Именно поэтому визуализация занимает центральное место в разведывательном анализе данных (EDA) и в представлении результатов исследования [3]. В курсе «Анализ данных в Python» для визуализации используются две основные библиотеки — Matplotlib и Seaborn [2].

Визуализация решает несколько задач. Во-первых, она помогает понять данные: распределения, взаимосвязи, выбросы, закономерности. Во-вторых, она позволяет проверить гипотезы: например, действительно ли выживаемость женщин выше, чем мужчин. В-третьих, она помогает представить результаты исследования: графики делают выводы наглядными и убедительными. В-четвёртых, она используется для отладки моделей: например, кривые обучения, ROC-кривые, матрицы ошибок[4].

Исторически визуализация данных развивалась вместе со статистикой. Ещё в XIX веке Флоренс Найтингейл использовала диаграммы для представления данных о смертности в армии. В XX веке Джон Тьюки включил визуализацию в разведывательный анализ данных. В XXI веке визуализация стала массовой благодаря библиотекам Python. Сегодня визуализация — это не только инструмент аналитика, но и средство коммуникации: графики используются в отчётах, презентациях, научных статьях, бизнес-аналитике[2].

Визуализация данных опирается на несколько принципов восприятия. Человеческий глаз лучше воспринимает положение и длину, чем площадь и

цвет. Поэтому линейные графики и столбчатые диаграммы часто эффективнее круговых диаграмм. Цвет должен помогать, а не отвлекать. Слишком много цветов делает график нечитаемым. Пропорции должны быть честными: обрезанная ось может преувеличить различия. Эти принципы лежат в основе эффективной визуализации.

Библиотека Matplotlib

Matplotlib — базовая библиотека Python для визуализации данных. Она предоставляет низкоуровневый контроль над графиками: можно настроить практически любой элемент — от цвета линии до положения подписей. Matplotlib лежит в основе многих других библиотек, включая Seaborn. Основные концепции Matplotlib — фигура (figure) и оси (axes). Фигура — это холст, на котором размещаются графики. Оси — это область, в которой строится график. На одной фигуре может быть несколько осей.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(10, 6))
ax.plot([1, 2, 3, 4], [1, 4, 9, 16]) #рисунок 1
ax.set_title('Простой линейный график')
ax.set_xlabel('X')
ax.set_ylabel('Y')
plt.show()
```

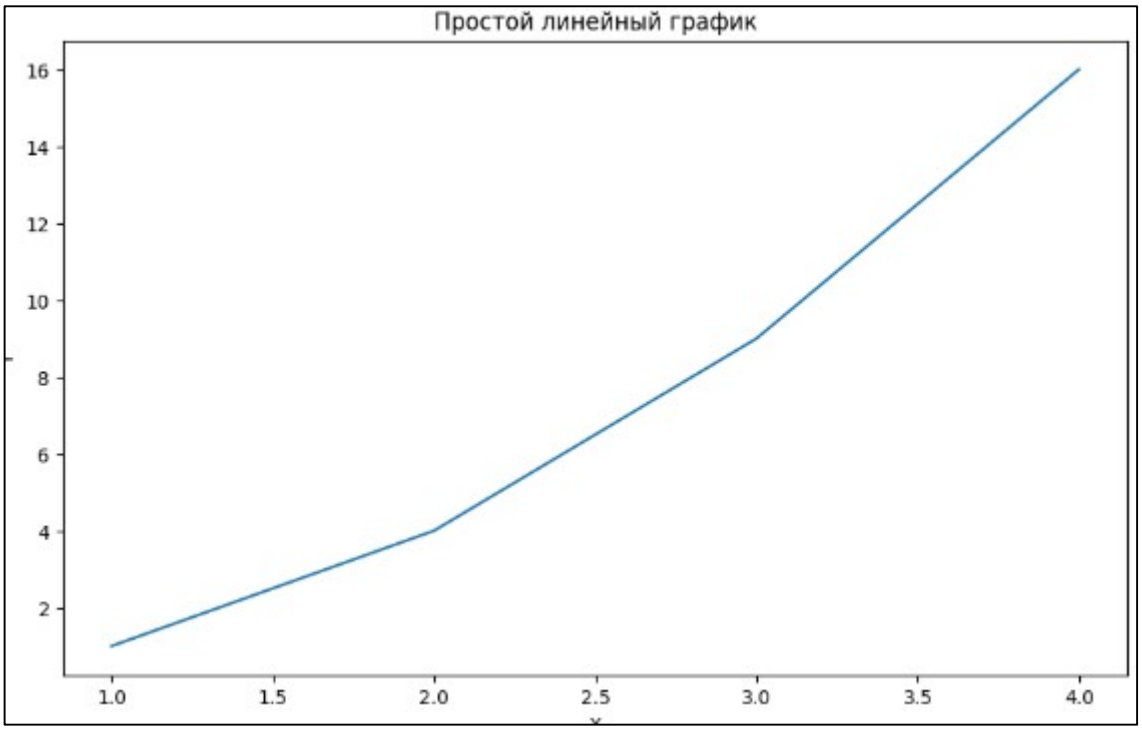


Рисунок 1 - Простой линейный график

Линейный график — самый простой тип визуализации. Он используется для отображения изменений во времени или зависимостей между переменными. Функция plot() принимает значения по оси X и Y, а также параметры стиля: цвет (color), тип линии (linestyle), толщину (linewidth), маркеры (marker). Можно строить несколько линий на одном графике, добавлять легенду через legend(). Легенда помогает различить линии, особенно если их несколько.

```
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot([1, 2, 3, 4], [1, 4, 9, 16], label='Квадраты', color='blue', marker='o')
ax.plot([1, 2, 3, 4], [1, 2, 3, 4], label='Линейная', color='red', linestyle='--')
ax.set_title('Сравнение функций') #рисунок 2
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.legend()
plt.show()
```

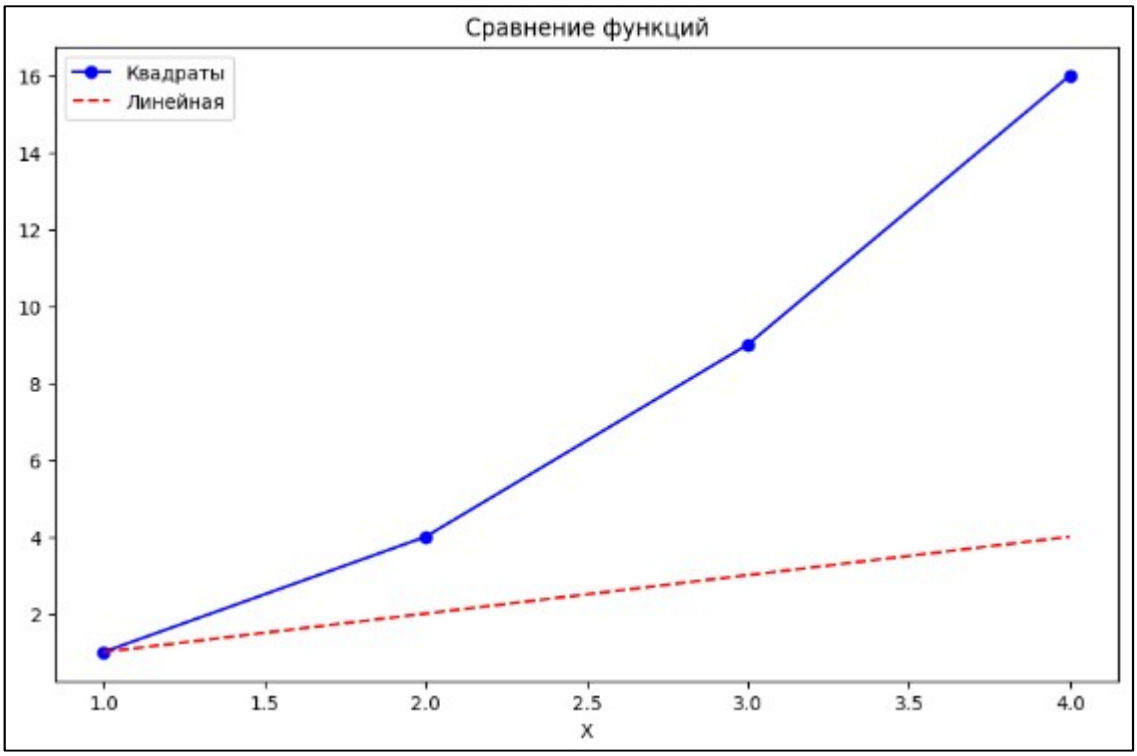


Рисунок 2 - Сравнение функций

Линейные графики часто используются для отображения временных рядов. Например, можно построить график изменения курса валюты, температуры, продаж. Для этого по оси X откладывается время, по оси Y — значение. Если временной ряд содержит сезонность или тренд, это видно на графике. Линейные графики также используются для отображения кривых обучения моделей машинного обучения: по оси X — эпохи, по оси Y — ошибка.

Столбчатые диаграммы используются для отображения категориальных данных. Функция bar() принимает категории и значения. Можно настроить цвет, ширину столбцов, подписи. Горизонтальные столбчатые диаграммы строятся функцией barh(). Они удобны, когда категорий много или их названия длинные. Столбчатые диаграммы часто используются для сравнения значений между категориями: например, средняя зарплата по отделам, количество продаж по регионам.

```
categories = ['A', 'B', 'C', 'D']
values = [10, 20, 15, 25]
```

```
fig, ax = plt.subplots(figsize=(8, 6))
ax.bar(categories, values, color='steelblue')
ax.set_title('Столбчатая диаграмма') #рисунок 3
ax.set_xlabel('Категории')
ax.set_ylabel('Значения')
plt.show()
```

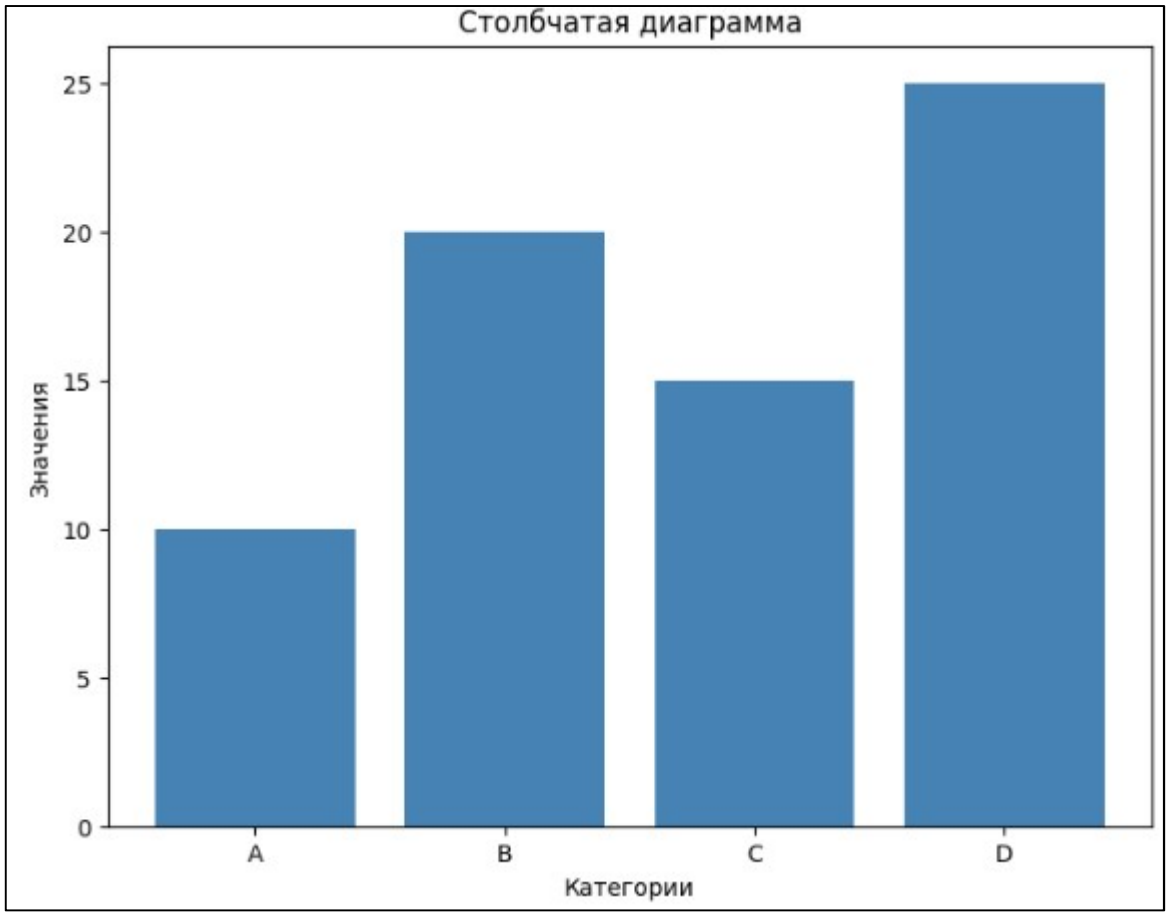


Рисунок 3 - Столбчатая диаграмма

Гистограммы используются для отображения распределения числовых данных. Функция hist() разбивает данные на интервалы (бины) и подсчитывает количество значений в каждом. Параметр bins задаёт число интервалов. Гистограмма позволяет увидеть форму распределения: нормальное, скошенное, бимодальное. Если распределение имеет два пика,

это может указывать на наличие двух групп в данных. Если распределение скошено вправо, это может указывать на наличие выбросов.

```
import numpy as np
data = np.random.randn(1000)
fig, ax = plt.subplots(figsize=(8, 6))
ax.hist(data, bins=30, color='skyblue', edgecolor='black')
ax.set_title('Гистограмма распределения') #рисунок 4
ax.set_xlabel('Значения')
ax.set_ylabel('Частота')
plt.show()
```

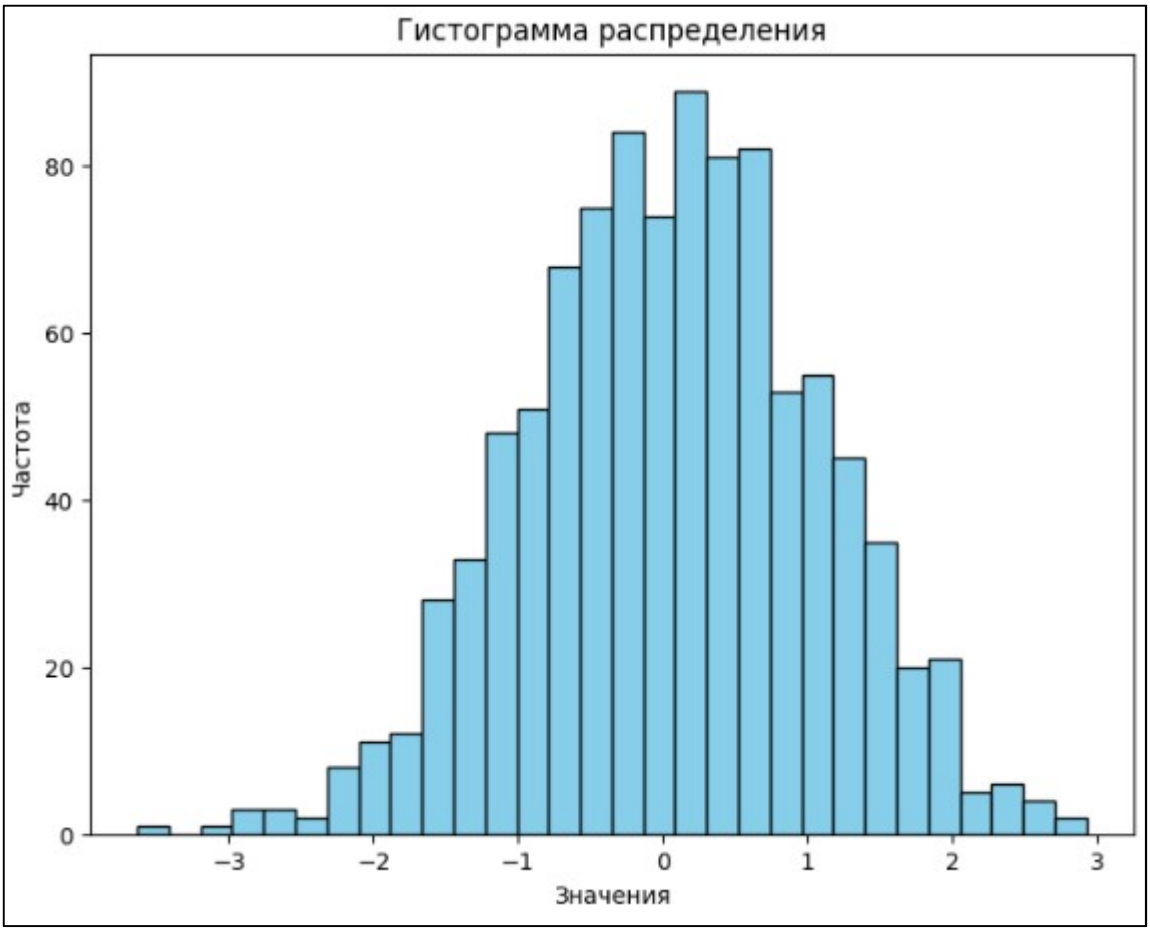


Рисунок 4 - Гистограмма распределения

Диаграммы рассеяния (scatter plot) используются для отображения взаимосвязи между двумя числовыми переменными.

Функция scatter() принимает значения X и Y, а также параметры размера (s), цвета (c), прозрачности (alpha). Диаграммы рассеяния позволяют увидеть корреляцию, кластеры, выбросы. Если точки образуют линейную зависимость, это указывает на корреляцию. Если точки образуют кластеры, это может указывать на наличие групп. Если есть точки, далеко отстоящие от основной массы, это выбросы.

```
x = np.random.randn(100)
y = 2 * x + np.random.randn(100)

fig, ax = plt.subplots(figsize=(8, 6))
ax.scatter(x, y, alpha=0.6, c='green')
ax.set_title('Диаграмма рассеяния') #рисунок 5
ax.set_xlabel('X')
ax.set_ylabel('Y')
plt.show()
```

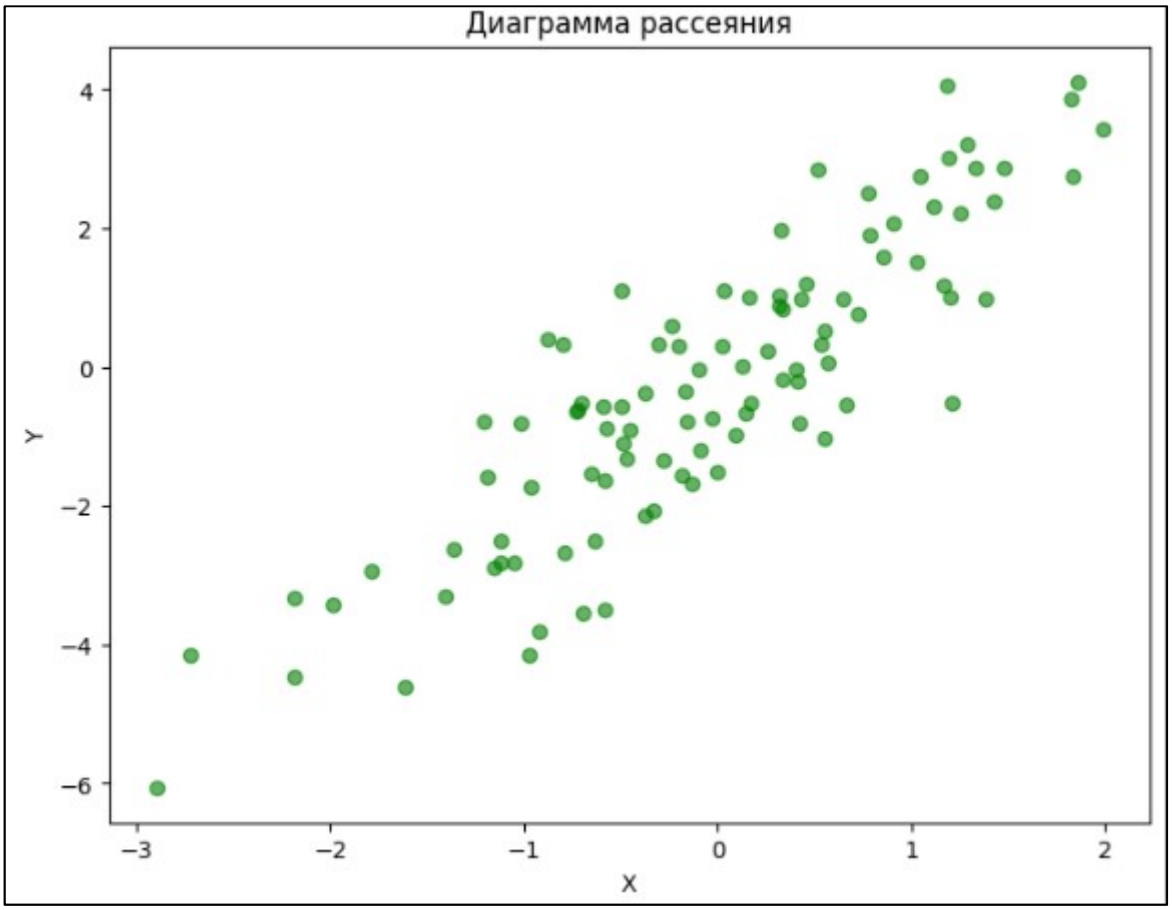


Рисунок 5 - Диаграмма рассеяния

Настройка графиков в Matplotlib включает множество параметров. Заголовок задаётся функцией set_title(), подписи осей — set_xlabel() и set_ylabel(). Легенда добавляется функцией legend(). Сетка включается функцией grid(). Ограничения осей задаются функциями set_xlim() и set_ylim(). Деления осей настраиваются через set_xticks() и set_yticks(). Сохранение графика в файл осуществляется функцией savefig(). Эти настройки позволяют сделать график информативным и красивым.

```
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot([1, 2, 3, 4], [1, 4, 9, 16])
ax.set_title('График с настройками', fontsize=14) #рисунок 6
ax.set_xlabel('X', fontsize=12)
ax.set_ylabel('Y', fontsize=12)
```

```
ax.grid(True, alpha=0.3)
ax.set_xlim(0, 5)
ax.set_ylim(0, 20)
plt.savefig('plot.png', dpi=300)
plt.show()
```

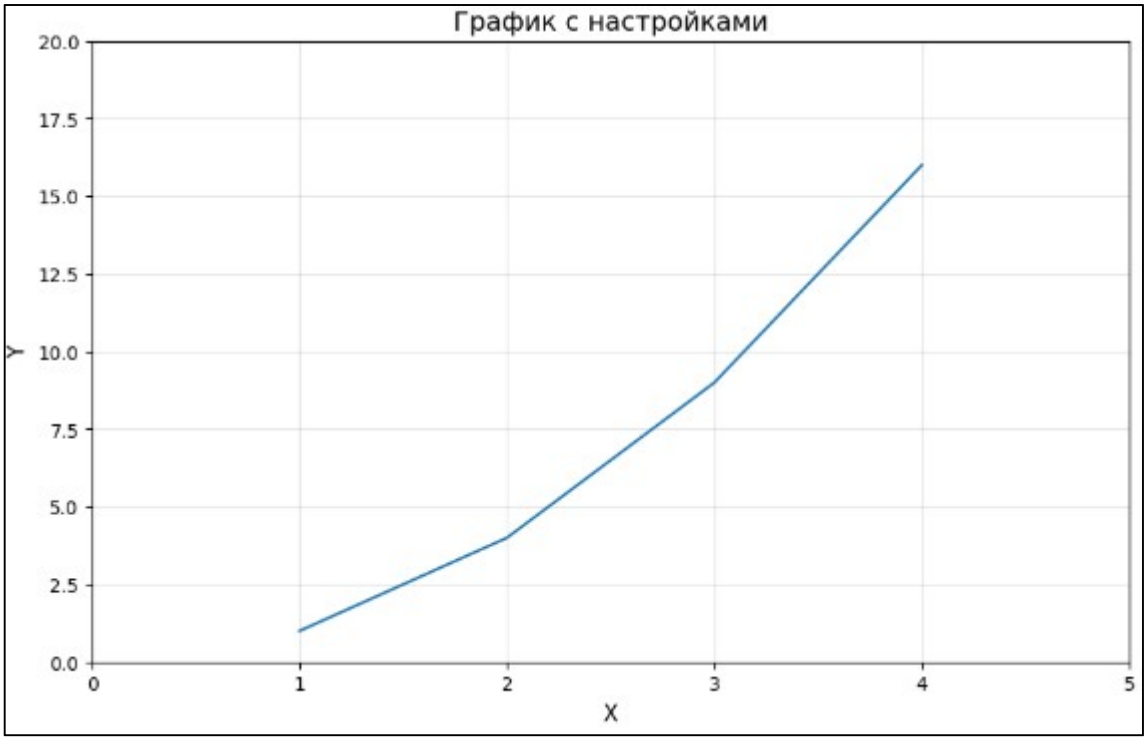


Рисунок 6 - График с настройками

Matplotlib также поддерживает подграфики (subplots). Функция subplots() позволяет создать сетку графиков. Параметры nrows и ncols задают число строк и столбцов. Каждый подграфик доступен через индексы. Это удобно для сравнения нескольких распределений или зависимостей. Например, можно построить четыре графика на одной фигуре: линейный, столбчатый, гистограмму и диаграмму рассеяния. Это позволяет увидеть разные аспекты данных в одном месте.

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes[0, 0].plot([1, 2, 3, 4], [1, 4, 9, 16])
axes[0, 0].set_title('Линейный')
```

```
axes[0, 1].bar(['A', 'B', 'C'], [10, 20, 15])
axes[0, 1].set_title('Столбчатый')
axes[1, 0].hist(np.random.randn(500), bins=20)
axes[1, 0].set_title('Гистограмма')
axes[1, 1].scatter(np.random.randn(50), np.random.randn(50))
axes[1, 1].set_title('Рассеяние')
plt.tight_layout()#рисунок 7
plt.show()
```

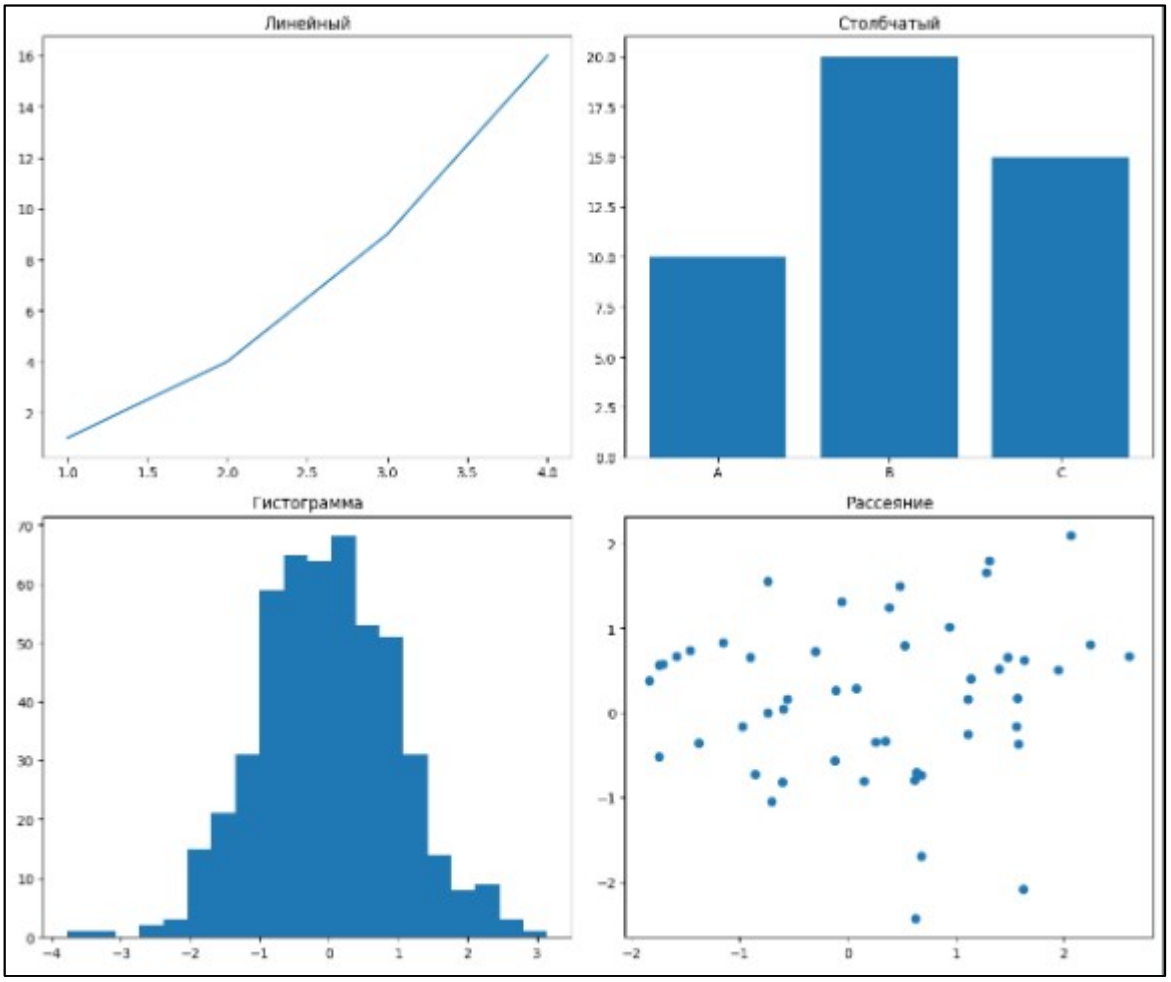


Рисунок 7 – Линейный, Столбчатый, Гистограмма, Рассеяние

Matplotlib также поддерживает аннотации. Функция annotate() позволяет добавить текст к графику с указанием точки. Это полезно для выделения важных значений или событий.

Функция text() добавляет текст в произвольное место графика. Аннотации помогают сделать график более информативным.

```
fig, ax = plt.subplots(figsize=(10, 6))
ax.plot([1, 2, 3, 4], [1, 4, 9, 16], marker='o')
ax.annotate('Максимум', xy=(4, 16), xytext=(3, 14),
            arrowprops=dict(arrowstyle='->'))
ax.set_title('График с аннотацией') #рисунок 8
plt.show()
```

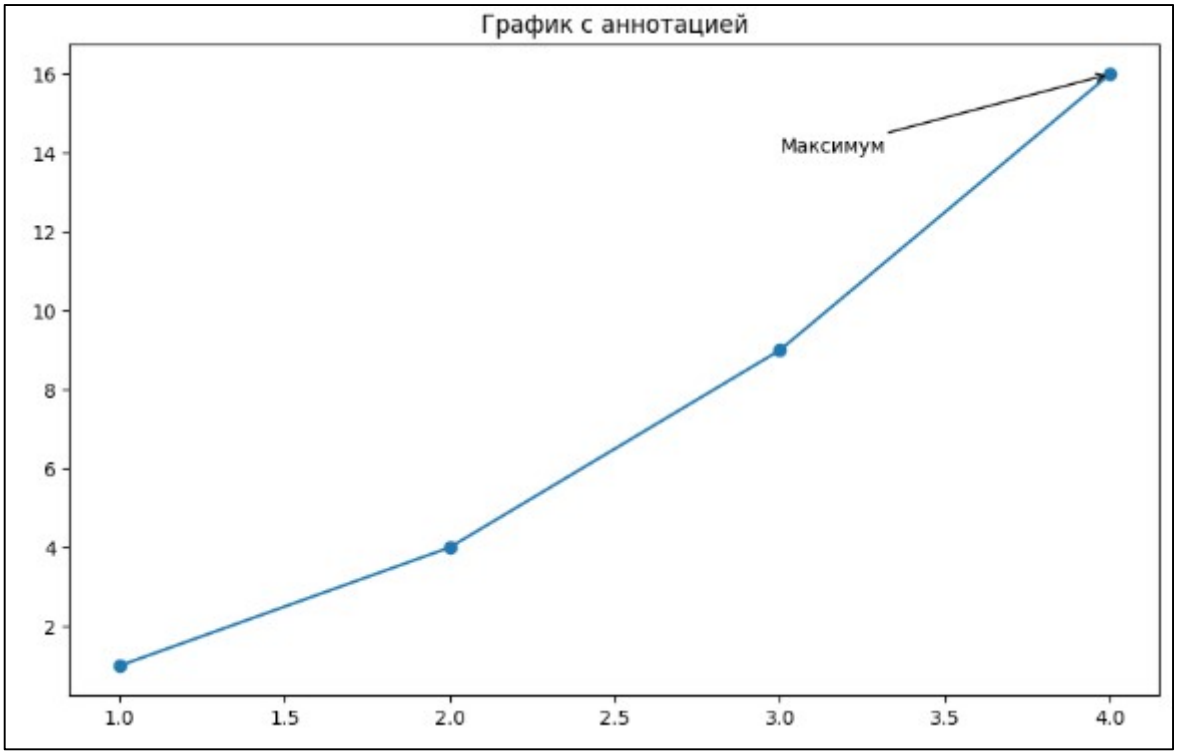


Рисунок 8 - График с аннотацией

Библиотека Seaborn

Seaborn — библиотека для статистической визуализации, построенная поверх Matplotlib. Она предоставляет более высокоуровневый интерфейс и красивые стили по умолчанию. Seaborn особенно удобен для визуализации распределений, взаимосвязей и категориальных данных. Seaborn

интегрируется с Pandas: многие функции принимают DataFrame и имена столбцов, что упрощает построение графиков.

Построение распределений осуществляется функциями histplot(), kdeplot(), displot(). histplot() строит гистограмму с возможностью добавления кривой плотности (KDE). kdeplot() строит только кривую плотности. displot() — это фигурный интерфейс для построения распределений. Эти функции позволяют сравнивать распределения между группами через параметр hue. Например, можно сравнить распределение возраста между выжившими и погибшими.

```
import seaborn as sns
import matplotlib.pyplot as plt
df = sns.load_dataset('titanic')
sns.histplot(data=df, x='age', hue='sex', kde=True)
plt.title('Распределение возраста по полу') #рисунок 9
plt.show()
sns.kdeplot(data=df, x='fare', hue='survived', fill=True)
plt.title('Распределение стоимости билета по выживаемости') #рисунок 9
plt.show()
```

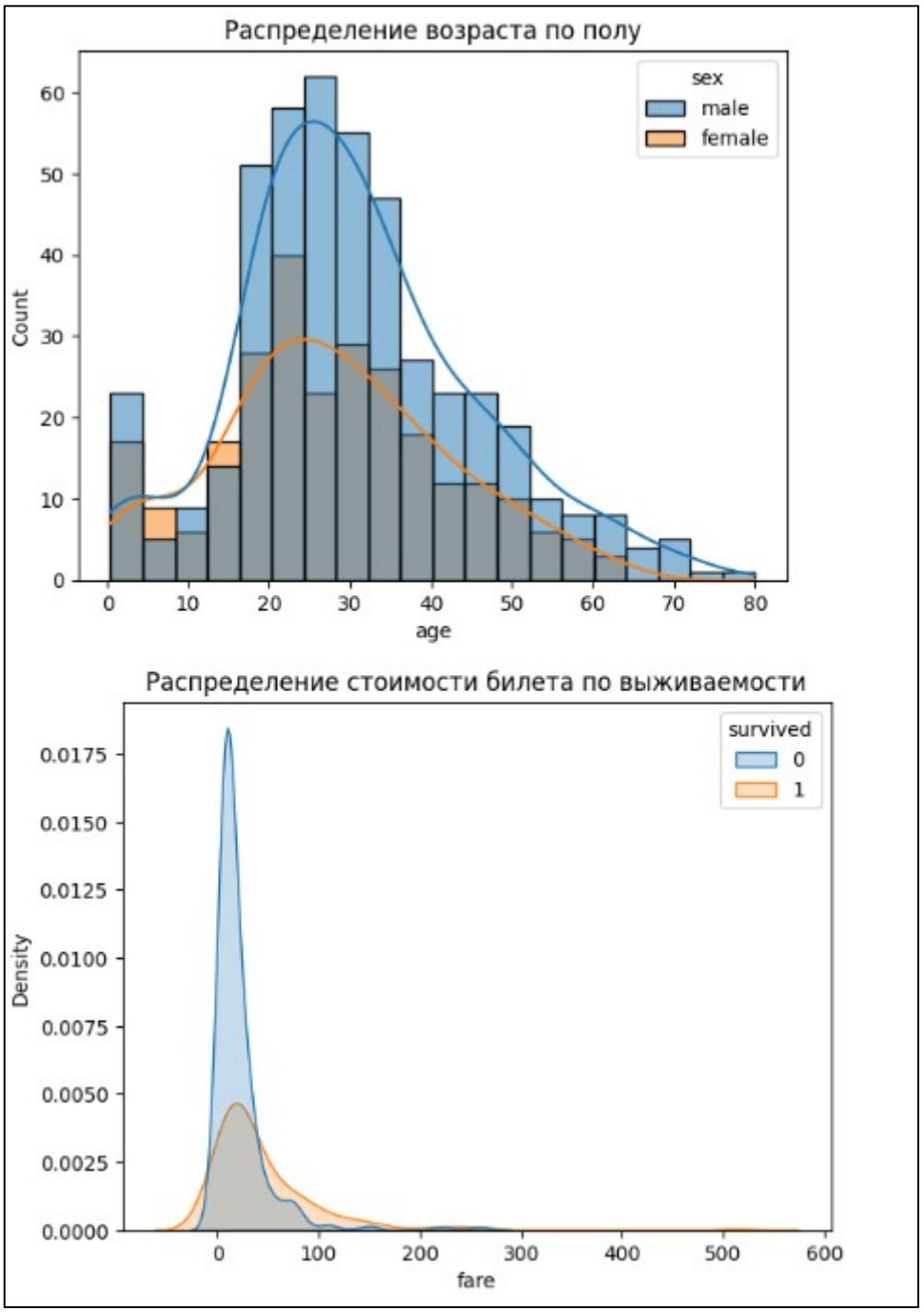


Рисунок 9 - Распределение возраста по полу и Распределение стоимости билета по выживаемости'

Отображение взаимосвязей между переменными осуществляется функциями scatterplot(), lineplot(), relplot(). scatterplot() строит диаграмму рассеяния с возможностью цветового кодирования по категории. lineplot() строит линейный график с доверительными интервалами. relplot() — это фигурный интерфейс для построения графиков

взаимосвязей. Эти функции позволяют увидеть, как связаны переменные и как на это влияют категории.

```
sns.scatterplot(data=df, x='age', y='fare', hue='survived', alpha=0.6)
plt.title('Взаимосвязь возраста и стоимости билета') #рисунок 10
plt.show()

sns.lineplot(data=df, x='age', y='fare', hue='sex')
plt.title('Зависимость стоимости билета от возраста') #рисунок 10
plt.show()
```

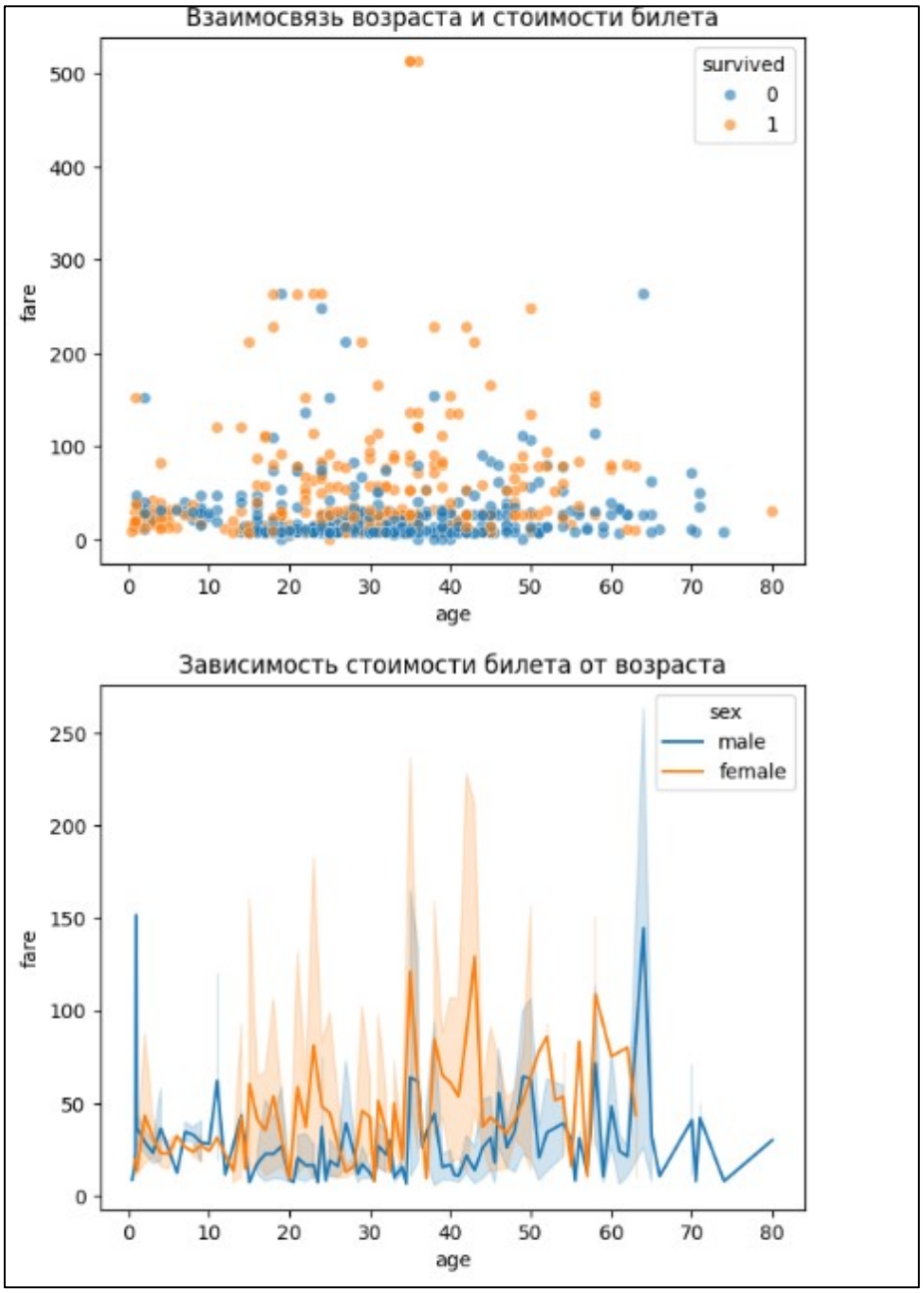


Рисунок 10 - Взаимосвязь возраста и стоимости билета и Зависимость стоимости билета от возраста

Сравнение категориальных данных осуществляется функциями boxplot(), violinplot(), barplot(), countplot(). boxplot() строит ящик с усами, показывающий медиану, квартили и выбросы. violinplot() объединяет box-plot с кривой плотности. barplot() строит столбчатую диаграмму со средними значениями и доверительными интервалами. countplot() подсчитывает количество наблюдений в каждой категории. Эти функции позволяют сравнить распределения между группами.

```
sns.boxplot(data=df, x='pclass', y='fare', hue='survived')
plt.title('Стоимость билета по классам и выживаемости') #рисунок 11
plt.show()

sns.violinplot(data=df, x='sex', y='age', hue='survived', split=True)
plt.title('Распределение возраста по полу и выживаемости') #рисунок 11
plt.show()

sns.countplot(data=df, x='sex', hue='survived')
plt.title('Выживаемость по полу') #рисунок 12
plt.show()
```

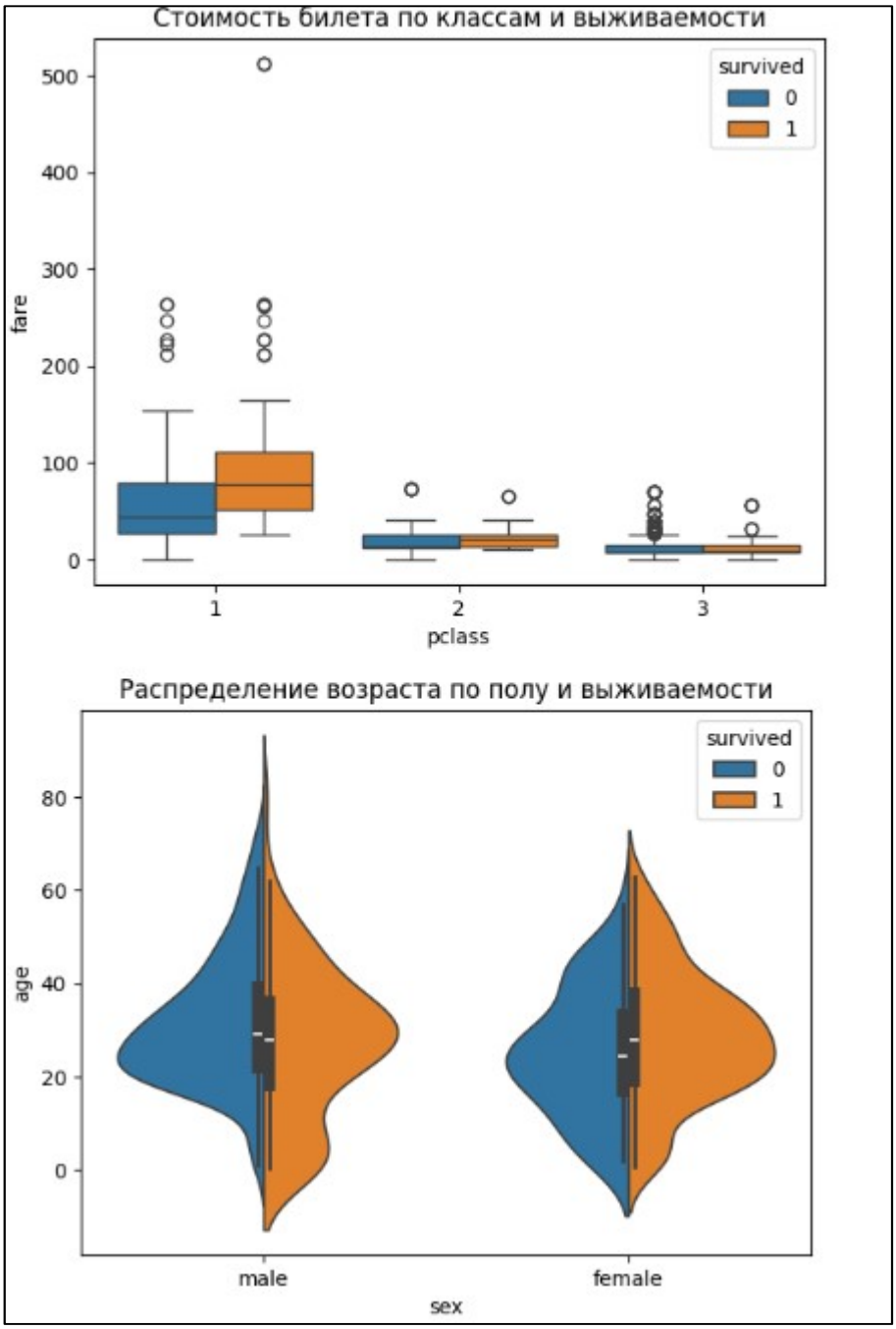


Рисунок 11 - Стоимость билета по классам и выживаемости и Распределение возраста по полу и выживаемости

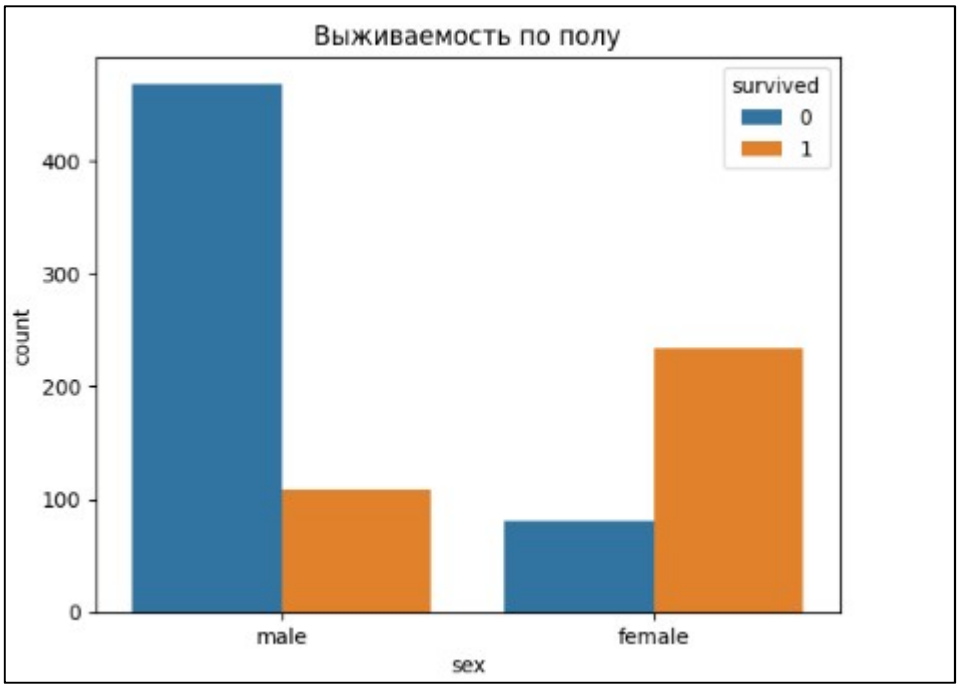


Рисунок 12 - Выживаемость по полу

Тепловые карты корреляций строятся функцией heatmap(). Она принимает матрицу корреляций и отображает её в виде цветовой карты. Параметр annot=True добавляет числовые значения в ячейки. Параметр cmap задаёт цветовую схему. Тепловые карты позволяют быстро увидеть, какие признаки связаны между собой. Красный цвет означает положительную корреляцию, синий — отрицательную, белый — отсутствие корреляции.

```
corr = df.corr(numeric_only=True)

sns.heatmap(corr, annot=True, cmap='coolwarm', center=0, fmt='.2f')
plt.title('Тепловая карта корреляций') #рисунок 13
plt.show()
```

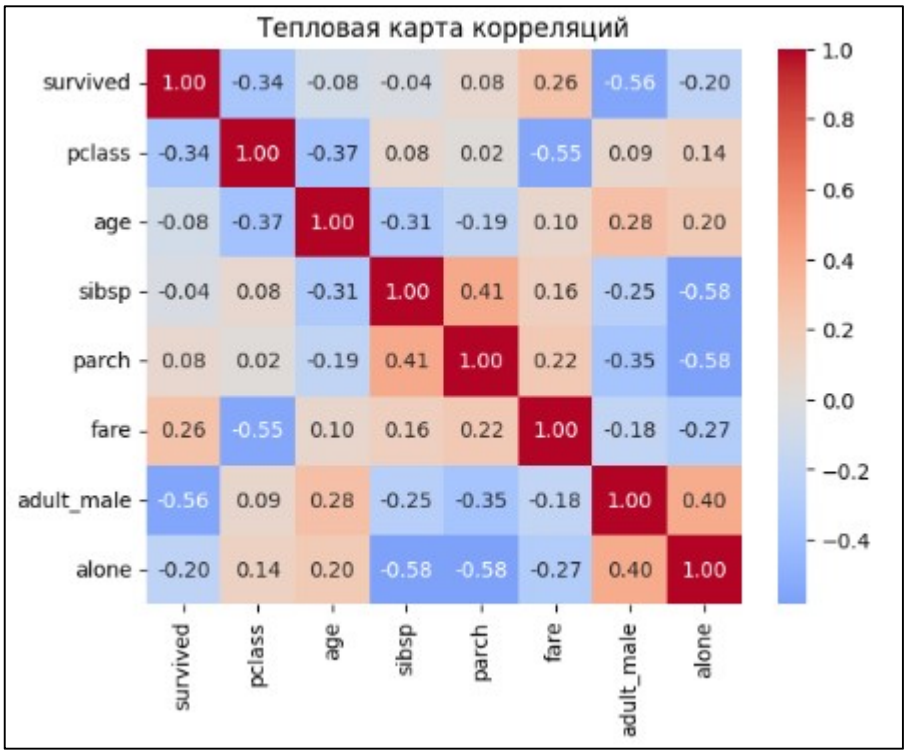


Рисунок 13 - Тепловая карта корреляций

Seaborn также предоставляет функции для парных графиков (pairplot()), которые строят матрицу рассеяния для всех числовых признаков. Это удобно для быстрого обзора взаимосвязей. Функция jointplot() строит совместное распределение двух переменных с маргинальными распределениями по осям. Эти функции позволяют увидеть общую картину данных.

```
sns.pairplot(df[['age', 'fare', 'pclass', 'survived']], hue='survived') #рисунок 14
plt.show()

sns.jointplot(data=df, x='age', y='fare', hue='survived') #рисунок 15
plt.show()
```

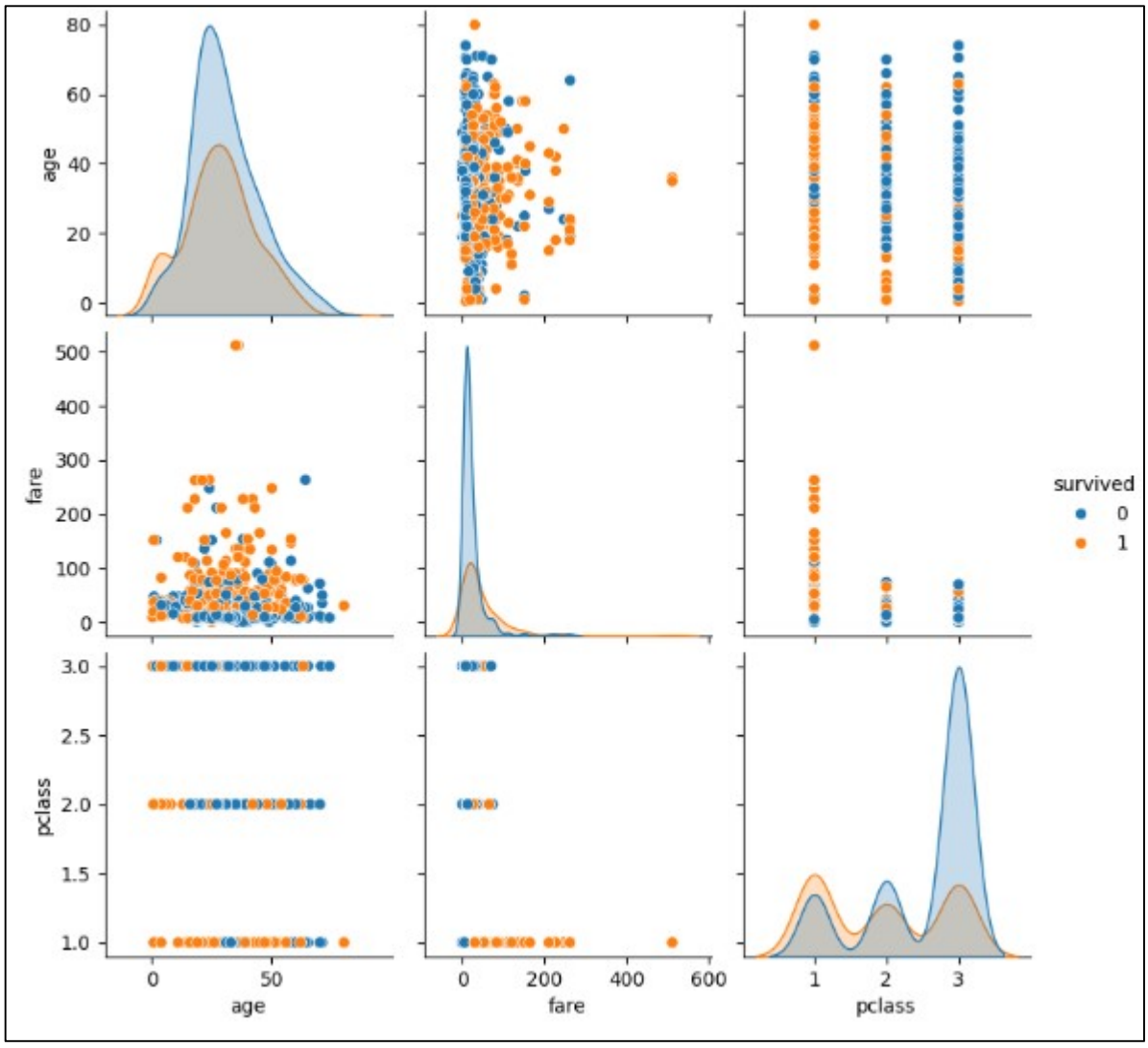


Рисунок 14 - Матрица диаграмм рассеяния

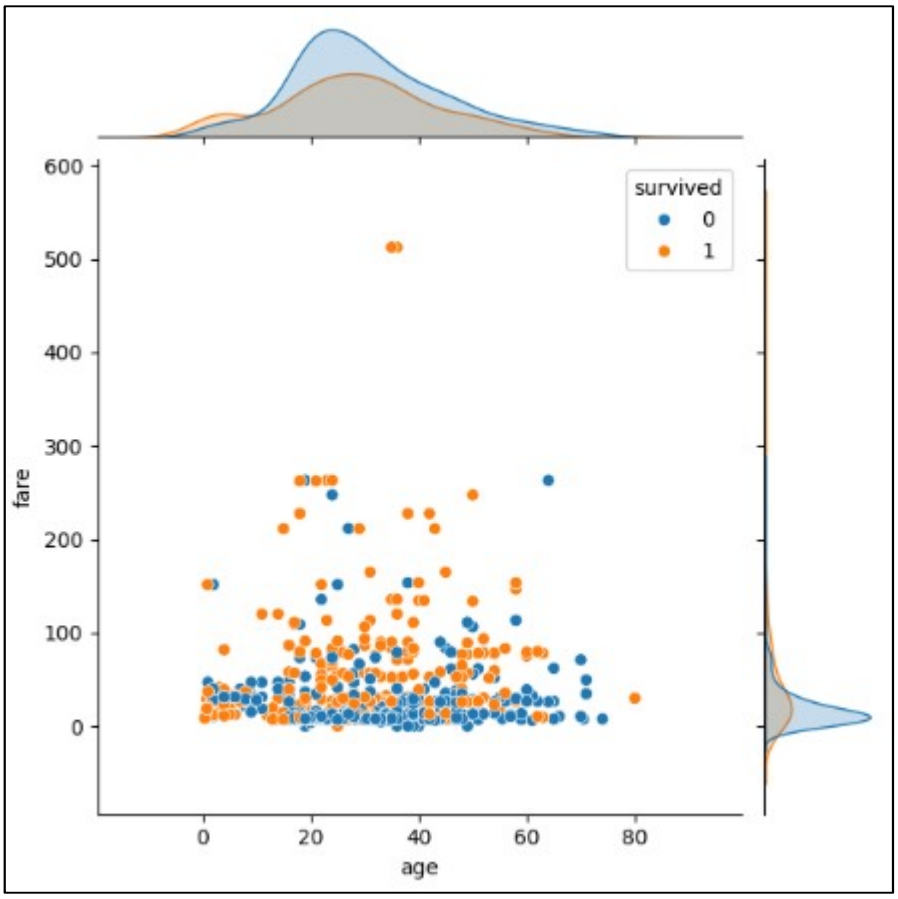


Рисунок 15 - Совместное распределение двух признаков

Seaborn поддерживает стили (set_style()) и палитры (set_palette()). Стили задают общий вид графиков: фон, сетку, шрифты. Палитры задают цветовые схемы. Это позволяет делать графики единообразными и красивыми. Стили: white, dark, whitegrid, darkgrid, ticks. Палитры: deep, muted, pastel, bright, dark, colorblind.

```
sns.set_style('whitegrid')
sns.set_palette('husl')

sns.boxplot(data=df, x='pclass', y='fare', hue='survived')
plt.title('Стоимость билета по классам и выживаемости') #рисунок 16
plt.show()
```

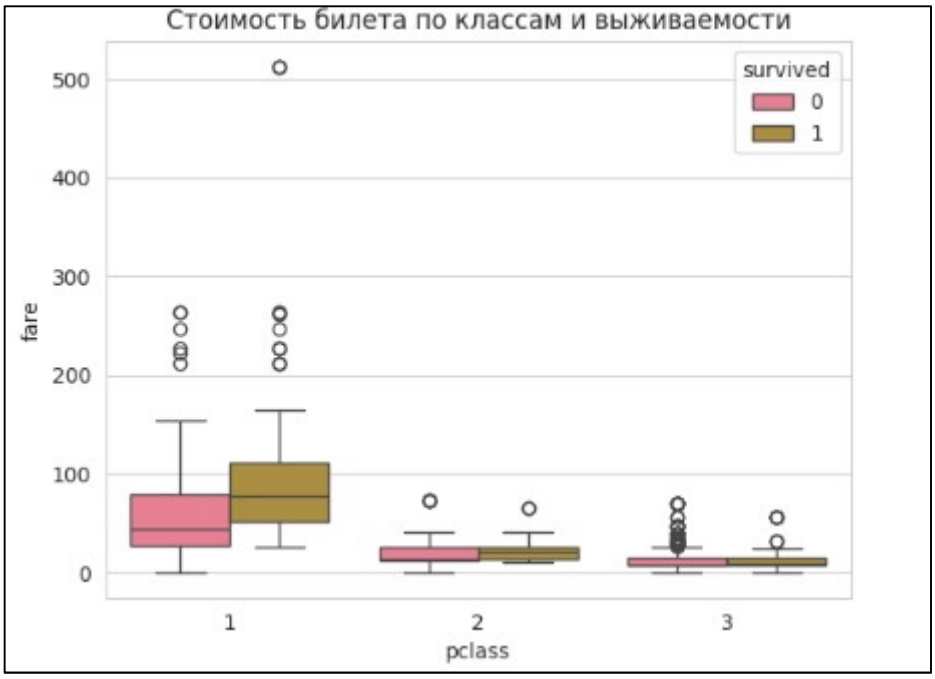


Рисунок 16 - Стоимость билета по классам и выживаемости

Seaborn также поддерживает регрессионные графики (regplot(), lmplot()). Они строят диаграмму рассеяния с линией регрессии. Это полезно для визуальной оценки линейной зависимости. Функция residplot() строит график остатков, который помогает проверить адекватность модели.

```
sns.regplot(data=df, x='age', y='fare')
plt.title('Регрессионная зависимость') #рисунок 17
plt.show()

sns.residplot(data=df, x='age', y='fare')
plt.title('График остатков') #рисунок 17
plt.show()
```

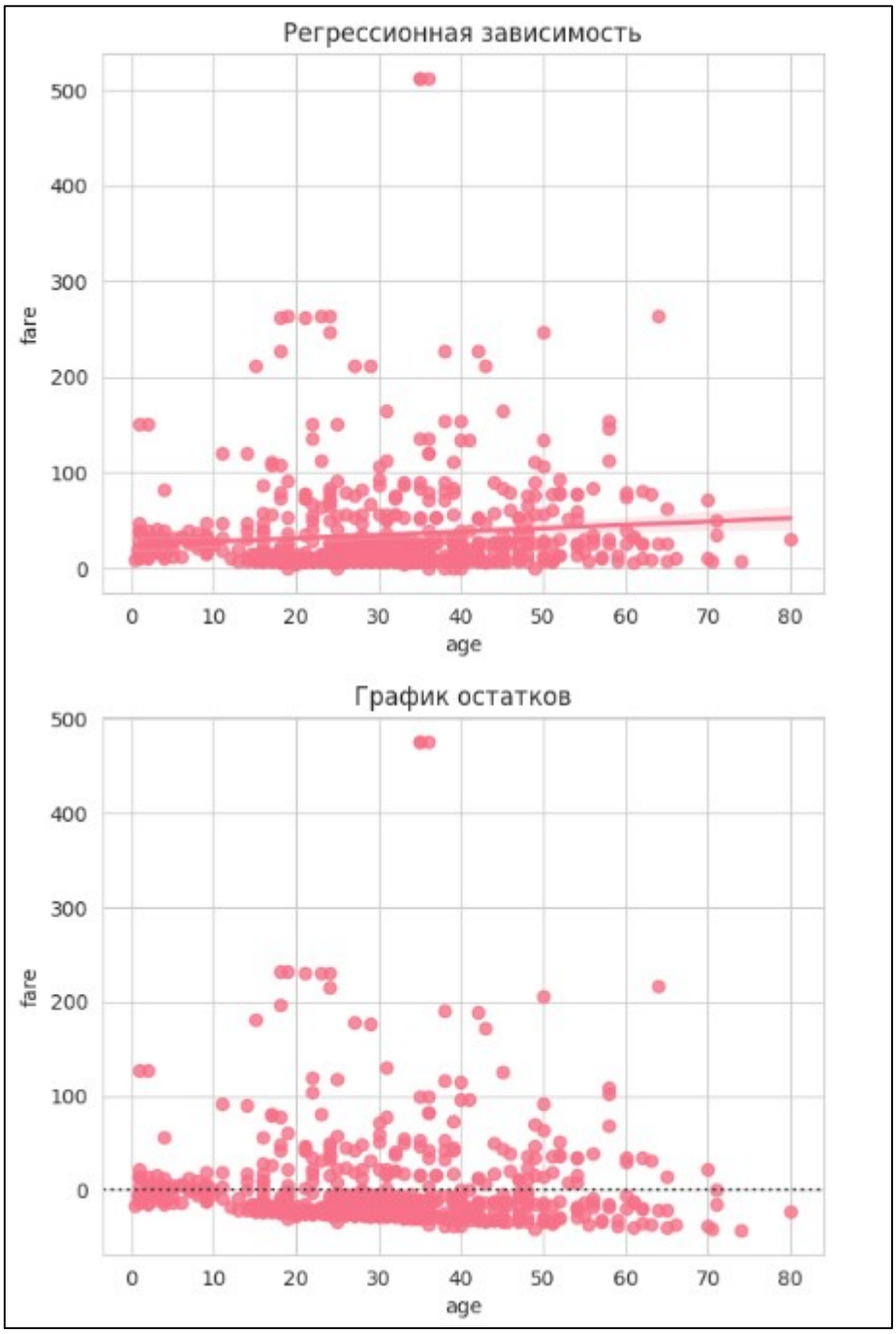


Рисунок 17 - Регрессионная зависимость и График остатков

Принципы эффективной визуализации

Эффективная визуализация — это не просто красивые графики, а графики, которые правильно передают информацию. Существует несколько принципов, которых стоит придерживаться.

Первый принцип — выбирать тип графика под задачу. Для распределений подходят гистограммы и box-plot. Для взаимосвязей —

диаграммы рассеяния и линейные графики. Для категориальных данных — столбчатые диаграммы и countplot. Для корреляций — тепловые карты. Неправильный выбор типа графика может исказить информацию. Например, круговая диаграмма плохо подходит для сравнения многих категорий.

Второй принцип — не перегружать график. Слишком много линий, цветов, подписей делают график нечитаемым. Лучше строить несколько простых графиков, чем один сложный. Если нужно показать много данных, можно использовать подграфики или интерактивные графики.

Третий принцип — подписывать оси и добавлять заголовок. График без подписей непонятен. Заголовок должен отражать суть графика, подписи осей — единицы измерения. Если график содержит несколько линий, нужна легенда.

Четвёртый принцип — использовать цвет осмысленно. Цвет должен помогать, а не отвлекать. Для категориальных данных подходят разные цвета, для числовых — градиенты. Слишком яркие цвета утомляют глаз. Для людей с дальтонизмом стоит использовать специальные палитры (colorblind).

Пятый принцип — избегать искажений. Например, обрезанная ось Y может преувеличить различия. Логарифмическая шкала может быть уместна для данных с большим разбросом. Круговые диаграммы плохо передают различия между близкими значениями.

Шестой принцип — учитывать аудиторию. Для специалистов подходят детальные графики, для широкой аудитории — простые и наглядные. В научной статье уместны строгие графики, в презентации — яркие и лаконичные.

Представление результатов исследования

Визуализация — это не только инструмент анализа, но и способ представления результатов. В отчёте или презентации графики должны быть

понятны без дополнительных пояснений. Для этого стоит придерживаться нескольких правил.

Первое правило — каждый график должен отвечать на конкретный вопрос. Не стоит строить графики «на всякий случай». Каждый график должен быть обоснован. Например, если исследуется выживаемость на Титанике, можно построить график выживаемости по полу, по классу, по возрасту. Каждый график отвечает на свой вопрос.

Второе правило — графики должны быть согласованы между собой. Единый стиль, единые цвета, единые обозначения делают отчёт целостным. Если на одном графике выжившие обозначены синим, на другом они тоже должны быть синими.

Третье правило — графики должны сопровождаться выводами. График без вывода — это просто картинка. Вывод должен объяснять, что видно на графике и что это значит. Например: «Из графика видно, что выживаемость женщин значительно выше, чем мужчин: около 75% против 20%».

Четвёртое правило — графики должны быть воспроизводимы. Код, который строит графики, должен быть сохранён. Это позволяет при необходимости перестроить график или изменить его. В Jupyter Notebook код и графики хранятся вместе, что удобно.

Пятое правило — графики должны быть сохранены в высоком качестве. Для печати подходит формат PNG с разрешением 300 dpi или векторные форматы (PDF, SVG). Векторные форматы позволяют масштабировать график без потери качества.

```
fig, ax = plt.subplots(figsize=(10, 6))
sns.boxplot(data=df, x='pclass', y='fare', hue='survived', ax=ax)
ax.set_title('Стоимость билета по классам и выживаемости', fontsize=14)
ax.set_xlabel('Класс каюты', fontsize=12)
ax.set_ylabel('Стоимость билета', fontsize=12)
plt.tight_layout()
```

```
plt.savefig('boxplot.png', dpi=300) #рисунок 18
plt.show()
```

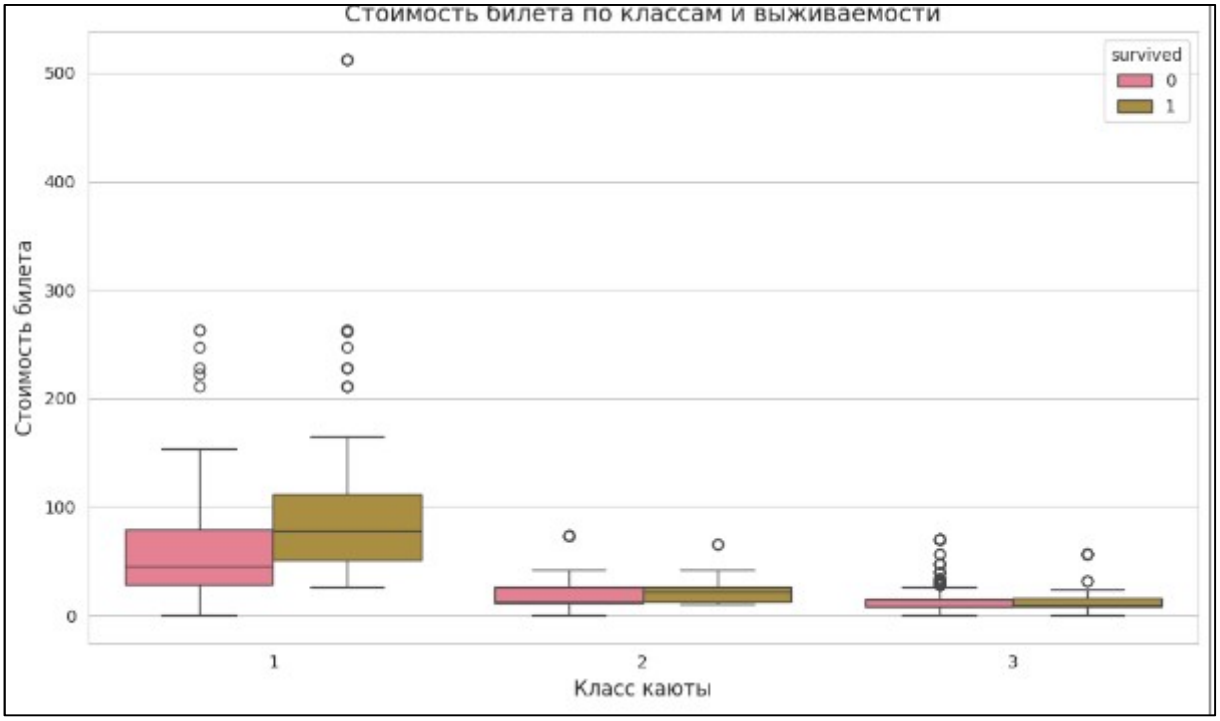


Рисунок 18 - Диаграмма размаха

Пример комплексного анализа с визуализацией

Рассмотрим пример комплексного анализа данных о Титанике с использованием визуализации. Сначала загрузим данные и проведём первичный анализ. Затем построим несколько графиков, каждый из которых отвечает на свой вопрос.

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

df = sns.load_dataset('titanic')

# Первичный анализ
```

```
print(df.info())
print(df.describe())
print(df.isnull().sum())

# Распределение возраста
sns.histplot(data=df, x='age', kde=True)
plt.title('Распределение возраста пассажиров')
plt.show()

# Выживаемость по полу
sns.countplot(data=df, x='sex', hue='survived')
plt.title('Выживаемость по полу')
plt.show()

# Выживаемость по классу
sns.countplot(data=df, x='pclass', hue='survived')
plt.title('Выживаемость по классу каюты')
plt.show()

# Стоимость билета по классам
sns.boxplot(data=df, x='pclass', y='fare')
plt.title('Стоимость билета по классам')
plt.show()

# Тепловая карта корреляций
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap='coolwarm', center=0)
plt.title('Тепловая карта корреляций')
plt.show()
```

Из этих графиков можно сделать несколько выводов. Во-первых, распределение возраста имеет пик в районе 20–30 лет, что соответствует молодым пассажирам. Во-вторых, выживаемость женщин значительно выше, чем мужчин. В-третьих, выживаемость пассажиров первого класса выше, чем второго и третьего. В-четвёртых, стоимость билета сильно варьируется, особенно в первом классе. В-пятых, корреляция между признаками показывает, что класс каюты и стоимость билета связаны отрицательно.

Выводы по лекции

В ходе лекции мы рассмотрели библиотеки Matplotlib и Seaborn для визуализации данных. Matplotlib предоставляет низкоуровневый контроль над графиками, Seaborn — высокоуровневый интерфейс и красивые стили. Мы изучили основные типы графиков: линейные, столбчатые, гистограммы, диаграммы рассеяния, box-plot, тепловые карты. Мы рассмотрели принципы эффективной визуализации и правила представления результатов исследования. Ключевая идея лекции состоит в том, что визуализация — это не просто построение графиков, а инструмент анализа и коммуникации, который помогает понять данные и представить результаты.

Контрольные вопросы:

1. Какие библиотеки можно использовать для визуализации данных в Python?
2. Как создать простую линейную графику с помощью Matplotlib?
3. Что такое диаграмма рассеяния и как она может помочь в анализе данных?
4. Какие типы графиков используются для отображения распределений данных?
5. Что такое тепловая карта корреляций и как её интерпретировать?

Список источников:

1. Анализ данных : учебник для вузов / В. С. Мхитарян [и др.] ; под ред. В. С. Мхитаряна. — Москва : Юрайт, 2022. — 490 с.

2. Вьюненко, Л. Ф. Анализ данных : учебник для вузов / Л. Ф. Вьюненко, Ю. Н. Гагарина, Н. Ю. Гагарина. — Москва : Юрайт, 2024. — 297 с.

3. Плас, Дж. Вандер. Python для сложных задач: наука о данных и машинное обучение. — Санкт-Петербург : Питер, 2022. — 576 с.

4. Грас, Дж. Data Science. Наука о данных с нуля. — Санкт-Петербург : БХВ-Петербург, 2021. — 336 с.